Started on	Monday, 16 March 2026, 10:01 PM
State	Finished
Completed on	Monday, 16 March 2026, 10:39 PM
Time taken	38 mins 13 secs
Grade	**400.00** out of 400.00 (**100**%)

Question **1**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Dunia *Middle-earth* dihuni oleh berbagai makhluk, masing-masing dengan sifat dan kemampuannya sendiri. Kamu diberikan kelas `Creature` (`.hpp` dan `.cpp`) beserta file `main.cpp`. Semua file ini **tidak boleh diubah**.

Tugasmu adalah **mendesain dan mengimplementasikan lima kelas turunan** (`Elf`, `Ranger`, `Maiar`, `Wizard`, dan `Balrog`) dengan menerapkan konsep **Inheritance** dan **Access Modifiers** (`public`, `protected`, atau `private`) yang tepat. **Pisahkan deklarasi (`.hpp`) dan implementasi (`.cpp`)** untuk setiap kelas. Kumpulkan dalam format `.zip` yang berisi hanya kelima kelas yang kamu implementasikan.

Spesifikasi Kelas

Elf

- Elf **adalah** (IS-A) `Creature`. Kode mana pun yang bekerja dengan `Creature` harus bisa menerima `Elf` secara langsung — upcast ke `Creature*` harus diperbolehkan dari luar kelas (misal dari main).
- Memiliki atribut tambahan `grace` (`int`).
- **Constructor**: menerima `name`, `hp`, `grace`.
- **`describe()`**: mencetak `"Elf [<name>] | HP: <hp> | Grace: <grace>\n"`
- **`getGrace()`**: mengembalikan nilai `grace`

Ranger

- Ranger **adalah** (IS-A) `Elf`, dan dengan demikian juga `Creature`. Upcast ke `Elf*` maupun `Creature*` harus diperbolehkan dari luar kelas. Ranger membawa `arrows` (`int`).
- **Constructor**: menerima `name`, `hp`, `grace`, `arrows`.
- **`describe()`**: mencetak `"Ranger [<name>] | HP: <hp> | Grace: <grace> | Arrows: <arrows>\n"`
- **`shoot()`**: mencetak `"<name> draws an arrow (grace: <grace>) and fires! Arrows left: <arrows-1>\n"`, lalu kurangi `arrows` sebanyak 1.

Maiar

- Maiar **menggunakan** (HAS-A) struktur `Creature` secara internal, ia punya nama dan HP, tetapi **tidak dikenal sebagai `Creature` dari luar kelas**. Kode eksternal **tidak boleh** melakukan upcast `Maiar*` ke `Creature*`. Namun, kelas-kelas yang mewarisi `Maiar` masih bisa mengakses atribut `Creature` (seperti `name` dan `hp`) secara internal. Maiar memiliki atribut `power` (`int`).
- **Constructor**: menerima `name`, `hp`, `power`, *constructor tidak bisa diinstansiasi langsung dari luar; hanya bisa dipanggil oleh subclass.*
- **`describe()`**: mencetak `"Maiar [<name>] | HP: <hp> | Power: <power>\n"`
- **`getPower()`**: mengembalikan nilai `power`

Wizard

- Wizard mewarisi `Maiar` dengan hubungan yang sepenuhnya tersembunyi dari luar kelas, baik dari kode eksternal **maupun dari subclass Wizard**. Tidak ada yang boleh upcast ke `Maiar*` atau `Creature*`, dan subclass Wizard pun tidak boleh mengakses atribut `Maiar` secara langsung. Wizard membawa `staffName` (`string`).
- **Constructor**: menerima `name`, `hp`, `power`, `staffName`.
- **`describe()`**: mencetak `"Wizard [<name>] | HP: <hp> | Staff: <staffName>\n"`
- **`cast()`**: mencetak `"<name> channels <power> power through <staffName>!\n"`

Balrog

- **Balrog** juga mewarisi **Maiar** dan dapat diakses dari luar kelas **Balrog** itu sendiri**.** Namun seperti semua Maiar, Balrog **tidak dikenal sebagai `Creature` dari luar**, kode eksternal tidak boleh upcast `Balrog*` ke `Creature*`. Balrog membawa `whipName` (`string`).
- **Constructor**: menerima `name`, `hp`, `power`, `whipName`.
- **`describe()`**: mencetak `"Balrog [<name>] | HP: <hp> | Power: <power> | Whip: <whipName>\n"`
- **`rage()`**: mencetak `"<name> cracks <whipName> with <power> power!\n"`

SARAN
- Tentukan mode inheritance setiap kelas sebelum menulis kode apapun. Implementasi kelas cukup mudah, cuma nguli aja ^ ^
- Jangan lupa untuk mendefinisikan file .hpp kalian, bisa dengan `#pragma once` atau `#ifndef` seperti di C
- Jangan lupa untuk tambahkan keyword override method describe dari Creature
- Has-A relationship di kelas Maiar bukanlah *Composition* melainkan bentuk use case dari salah satu mode inheritance

C++14

soal1.zip

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	20	Accepted	0.00 sec, 2.97 MB
2	20	Accepted	0.00 sec, 2.93 MB
3	20	Accepted	0.00 sec, 2.97 MB
4	20	Accepted	0.00 sec, 3.01 MB
5	20	Accepted	0.00 sec, 2.93 MB

Question **2**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Sebuah toko hewan sedang membangun sistem digital untuk mengelola hewan-hewan peliharaan yang dijualnya. Toko ini menjual tiga jenis hewan: **Kucing (Cat)**, **Anjing (Dog)**, dan **Burung Nuri (Parrot)**.

Semua hewan memiliki sifat dasar yang sama, **nama**, **umur**, dan **tingkat kebahagiaan**, tetapi setiap jenis hewan memiliki perilaku yang berbeda-beda saat bermain, makan, dan bersuara. Sistem ini harus mampu mengelola berbagai jenis hewan dengan konsep *polymorphism* melalui pointer ke base class.

Tugasmu adalah mengimplementasikan kelas turunan `Cat`, `Dog`, dan `Parrot` berdasarkan kelas dasar abstrak `Pet` yang sudah disediakan.

Untuk membantu Anda, berikut diberikan file-file yang **tidak boleh dirubah**.

- Pet.hpp: Header kelas dasar `Pet` (abstract class)
- Pet.cpp: Implementasi kelas dasar `Pet`
- Cat.hpp: Header kelas turunan `Cat`
- Dog.hpp: Header kelas turunan `Dog`
- Parrot.hpp: Header kelas turunan `Parrot`
- main.cpp: Program utama (driver)

Anda perlu mengimplementasikan kelas-kelas turunan `Cat`, `Dog`, dan `Parrot` dengan mengikuti spesifikasi berikut:

1. Kelas Cat

Kelas turunan dari `Pet` yang merepresentasikan kucing. Memiliki sifat tambahan yang menentukan apakah kucing tersebut kucing rumahan atau kucing luar.

- **Cat(string name, int age, bool indoor)** Constructor harus memanggil constructor `Pet` terlebih dahulu menggunakan initializer list, kemudian menginisialisasi atribut `indoor`. Setelah itu mencetak: `[CAT] {name} entered as indoor cat` jika indoor bernilai true, atau `[CAT] {name} entered as outdoor cat` jika false.
- **~Cat()** Destructor yang mencetak: `[CAT] {name} meows goodbye`
- **void makeSound() const override** Mengimplementasikan suara kucing. Mencetak: `[SOUND] {name}: Meow!`
- **void play() override** Meng-override perilaku bermain default. Perilaku bergantung pada tipe kucing:
  - Jika `indoor`: menambah `happiness` sebesar **15** dan mencetak: `[PLAY] {name} plays with yarn! Happiness: {happiness}`
  - Jika `outdoor`: menambah `happiness` sebesar **20** dan mencetak: `[PLAY] {name} chases mice! Happiness: {happiness}`
- **void feed() override** Meng-override perilaku makan default. Menambah `happiness` sebesar **8**. Mencetak: `[FEED] {name} eats fish! Happiness: {happiness}`

2. Kelas Dog

Kelas turunan dari `Pet` yang merepresentasikan anjing. Memiliki atribut tambahan yang menentukan apakah anjing tersebut sudah terlatih atau belum.

- **Dog(string name, int age, bool trained)** Constructor harus memanggil constructor `Pet` terlebih dahulu menggunakan initializer list, kemudian menginisialisasi atribut `trained`. Setelah itu mencetak: `[DOG] {name} entered as trained dog` jika trained bernilai true, atau `[DOG] {name} entered as untrained dog` jika false.
- **~Dog()** Destructor yang mencetak: `[DOG] {name} barks goodbye`
- **void makeSound() const override** Mengimplementasikan suara anjing. Mencetak: `[SOUND] {name}: Woof!`
- **void play() override** Meng-override perilaku bermain default. Perilaku bergantung pada status latihan:
  - Jika `trained`: menambah `happiness` sebesar **25** dan mencetak: `[PLAY] {name} fetches the ball! Happiness: {happiness}`
  - Jika `untrained`: menambah `happiness` sebesar **10** dan mencetak: `[PLAY] {name} runs around! Happiness: {happiness}`

3. Kelas Parrot

Kelas turunan dari `Pet` yang merepresentasikan burung nuri. Memiliki atribut tambahan yang menyimpan jumlah kata yang diketahui sang burung.

- **Parrot(string name, int age, int vocabulary)** Constructor harus memanggil constructor `Pet` terlebih dahulu menggunakan initializer list, kemudian menginisialisasi atribut `vocabulary`. Setelah itu mencetak: `[PARROT] {name} entered knowing {vocabulary} words`
- **~Parrot()** Destructor yang mencetak: `[PARROT] {name} squawks goodbye`
- **void makeSound() const override** Mengimplementasikan suara burung nuri. Mencetak: `[SOUND] {name}: Squawk! I know {vocabulary} words!`
- **void play() override** Meng-override perilaku bermain default. Saat bermain, burung nuri belajar kata baru: menambah `vocabulary` sebesar **1** dan menambah `happiness` sebesar **10**. Mencetak: `[PLAY] {name} learns a new word! Vocabulary: {vocabulary}, Happiness: {happiness}`
- **void feed() override** Meng-override perilaku makan default. Menambah `happiness` sebesar **3**. Mencetak: `[FEED] {name} eats seeds! Happiness: {happiness}`

Contoh Masukan dan Keluaran

Masukan:

```
9
1 1 Milo 3 1
1 2 Buddy 5 0
1 3 Rio 2 5
2 Milo
2 Buddy
2 Rio
5 Milo
5 Buddy
5 Rio
```

Keluaran:

```
[PET] Milo (age 3) entered the store
[CAT] Milo entered as indoor cat
[PET] Buddy (age 5) entered the store
[DOG] Buddy entered as untrained dog
[PET] Rio (age 2) entered the store
[PARROT] Rio entered knowing 5 words
[SOUND] Milo: Meow!
[SOUND] Buddy: Woof!
[SOUND] Rio: Squawk! I know 5 words!
[STATUS] Milo | Age: 3 | Happiness: 50
[STATUS] Buddy | Age: 5 | Happiness: 50
[STATUS] Rio | Age: 2 | Happiness: 50
[CAT] Milo meows goodbye
[PET] Milo left the store
[DOG] Buddy barks goodbye
[PET] Buddy left the store
[PARROT] Rio squawks goodbye
[PET] Rio left the store
```

> **Perhatikan:** Destructor kelas turunan dipanggil terlebih dahulu, baru kemudian destructor `Pet`. Ini karena urutan pemanggilan destructor dalam inheritance berjalan terbalik dari urutan constructor. Seluruh hewan yang masih tersisa di akhir program juga akan dihapus (destructor terpanggil).

C++14

soal2.zip

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	20	Accepted	0.00 sec, 2.99 MB
2	20	Accepted	0.00 sec, 2.96 MB
3	20	Accepted	0.00 sec, 2.93 MB
4	20	Accepted	0.00 sec, 2.93 MB
5	20	Accepted	0.00 sec, 2.91 MB

Question **3**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Kamu diminta mendesain dan mengimplementasikan sistem todo app sederhana. Aplikasi ini mengelola daftar aktivitas yang bisa di-*complete* untuk mendapatkan poin. Ada dua jenis aktivitas: **tugas sekali jalan** dan **kebiasaan harian**.

File `main.cpp` sudah disediakan dan **tidak boleh diubah**. Tugasmu adalah merancang kelas-kelas yang dibutuhkan agar `main.cpp` dapat berjalan dengan benar.

Perhatikan bagaimana `main.cpp` menggunakan kelas-kelasmu — dari situ kamu bisa menyimpulkan bagaimana hubungan antar kelas harus dirancang.

Kelas yang Harus Dibuat

1. `Activity`

`Activity` adalah representasi umum dari sebuah aktivitas. Kelas ini **tidak seharusnya bisa dibuat objeknya secara langsung**, karena perilaku spesifik (cara menyelesaikan aktivitas dan cara menampilkan status) bergantung pada jenis aktivitasnya.

Setiap aktivitas menyimpan nama dalam variabel `name` (bertipe `std::string`). Data ini perlu bisa diakses oleh kelas turunannya.

`Activity` memiliki tanggung jawab berikut:

- **Dibuat** dengan menerima sebuah nama.
- **Mengembalikan namanya** melalui `getName()`.
- **Menyelesaikan aktivitas** melalui `complete()`, yang mengembalikan jumlah poin (`int`) yang didapat. Namun, karena cara menyelesaikan tiap jenis aktivitas berbeda, `Activity` sendiri tidak tahu bagaimana melakukannya.
- **Menampilkan status** melalui `getStatus()`, yang mengembalikan `std::string`. Sama seperti `complete()`, formatnya bergantung pada jenis aktivitas.
- **Dihancurkan dengan benar** meskipun dihapus melalui pointer bertipe `Activity*`.

Pikirkan: jika `main.cpp` menyimpan semua aktivitas dalam array `Activity*` lalu memanggil `delete`, apa yang harus kamu pastikan agar destructor yang tepat terpanggil?

2. `TodoItem`

`TodoItem` adalah jenis aktivitas berupa tugas yang hanya perlu diselesaikan **satu kali**.

`TodoItem` menyimpan status apakah tugas sudah dikerjakan atau belum dalam variabel `isDone` (bertipe `bool`), yang awalnya bernilai `false`. Data ini hanya dikelola oleh `TodoItem` sendiri.

Tanggung jawab `TodoItem`:

- **Dibuat** dengan menerima sebuah nama.
- **Menyelesaikan tugas** (`complete()`): Jika belum selesai, tandai sebagai selesai dan berikan **10 poin**. Jika sudah pernah diselesaikan, berikan **0 poin**.
- **Menampilkan status** (`getStatus()`): Menghasilkan format `[TODO] <name> - Selesai` atau `[TODO] <name> - Belum`.
- **Saat dihancurkan**, mencetak `Menghapus TodoItem <name>` diikuti newline.

3. `DailyHabit`

`DailyHabit` adalah jenis aktivitas berupa kebiasaan yang bisa dilakukan **berkali-kali**, dengan poin yang semakin besar seiring frekuensi pengerjaannya.

`DailyHabit` menyimpan jumlah kali kebiasaan ini dilakukan dalam variabel `streak` (bertipe `int`), yang awalnya `0`. Data ini hanya dikelola oleh `DailyHabit` sendiri.

Tanggung jawab `DailyHabit`:

- **Dibuat** dengan menerima sebuah nama.
- **Menyelesaikan kebiasaan** (`complete()`): Tambah `streak` sebanyak 1, lalu berikan poin sebesar **10 × streak**. (Complete pertama = 10, kedua = 20, ketiga = 30, dst.)
- **Menampilkan status** (`getStatus()`): Menghasilkan format `[HABIT] <name> - Streak: <streak>`.
- **Saat dihancurkan**, mencetak `Menghapus DailyHabit <name>` diikuti newline.

Format Input

```
N
TYPE_1 NAME_1
TYPE_2 NAME_2
...
TYPE_N NAME_N
M
X_1
X_2
...
X_M
```

- `N` — jumlah aktivitas
- `TYPE` — jenis aktivitas: `TODO` atau `HABIT`
- `NAME` — nama aktivitas (satu kata, tanpa spasi)
- `M` — jumlah aksi complete
- `X` — indeks aktivitas yang di-complete (1-indexed)

Format Output

Untuk setiap aksi complete:

```
Mendapat <poin> poin dari <nama>
```

Di akhir program:

```
--- DASHBOARD AKHIR ---
<status tiap aktivitas>
Total Poin: <total>
<output destructor tiap aktivitas>
```

Contoh

Input

```
3
TODO KembalikanPetir
HABIT MeditasiOlympus
TODO BersihkanAegis
5
1
2
2
1
3
```

Output

```
Mendapat 10 poin dari KembalikanPetir
Mendapat 10 poin dari MeditasiOlympus
Mendapat 20 poin dari MeditasiOlympus
Mendapat 0 poin dari KembalikanPetir
Mendapat 10 poin dari BersihkanAegis
--- DASHBOARD AKHIR ---
[TODO] KembalikanPetir - Selesai
[HABIT] MeditasiOlympus - Streak: 2
[TODO] BersihkanAegis - Selesai
Total Poin: 50
Menghapus TodoItem KembalikanPetir
Menghapus DailyHabit MeditasiOlympus
Menghapus TodoItem BersihkanAegis
```

Penjelasan

1. `KembalikanPetir` (TODO) di-complete → belum selesai, dapat **10** poin
2. `MeditasiOlympus` (HABIT) complete pertama → streak = 1, dapat **10** poin
3. `MeditasiOlympus` (HABIT) complete kedua → streak = 2, dapat **20** poin
4. `KembalikanPetir` (TODO) dicoba lagi → sudah selesai, dapat **0** poin
5. `BersihkanAegis` (TODO) di-complete → belum selesai, dapat **10** poin

Total: 10 + 10 + 20 + 0 + 10 = **50**

File yang Disediakan

- `main.cpp` — **JANGAN DIUBAH (untuk testing)**.
- `Activity.hpp` (Perlu dilengkapi)
- `TodoItem.hpp` (Perlu dilengkapi)

- `DailyHabit.hpp` (Perlu dilengkapi)

File yang Harus Dikumpulkan

- `Activity.hpp`
- `TodoItem.hpp` & `TodoItem.cpp`
- `DailyHabit.hpp` & `DailyHabit.cpp`

C++14

soal3.zip

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	20	Accepted	0.00 sec, 3.02 MB
2	20	Accepted	0.00 sec, 3.01 MB
3	20	Accepted	0.00 sec, 3.13 MB
4	20	Accepted	0.00 sec, 3.01 MB
5	20	Accepted	0.00 sec, 3.02 MB

Question **4**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Anda diberikan kelas `Character` (kelas dasar/base class) beserta kelas **"cucu"** (`Paladin`) dan file `main.cpp`. Semua file ini **tidak boleh diubah**. File Soal

Diberikan pula expected output dari main.

Hirarki pewarisan yang digunakan membentuk pola **Diamond**:

```
        Character
       /         \
  Warrior       Mage
       \         /
       Paladin
```

`Paladin` mewarisi **kedua** kelas `Warrior` dan `Mage` secara langsung. Kedua kelas tersebut sama-sama turunan dari `Character`.

Tugas Anda adalah **mengimplementasikan dua kelas "anak"** yang menjadi penghubung antara `Character` dan kelas cucu:

- `Warrior` — menghubungkan `Character` ↔ `Paladin` (sisi kiri diamond)
- `Mage` — menghubungkan `Character` ↔ `Paladin` (sisi kanan diamond)

Anda harus menentukan sendiri **mode pewarisan yang tepat** agar:

Ketentuan yang harus dipenuhi

- `Character` hanya ter-instansiasi **satu kali** pada objek `Paladin` (tidak terduplikasi)
- Constructor `Character` hanya dipanggil **sekali** saat membuat `Paladin`
- Method `getStatus()`, `getCharacterId()`, dan `levelUp()` dari `Character` dapat dipanggil dari `Paladin` **tanpa ambiguitas**
- Destructor dipanggil dalam urutan: Paladin → Mage → Warrior → Character (satu kali)

`Warrior` (penghubung `Character` ↔ `Paladin`):

- Method: `void attack() const` dan `int getStrength() const`

`Mage` (penghubung `Character` ↔ `Paladin`):

- Method: `void castSpell() const` dan `int getMana() const`

Kumpulkan Warrior.cpp, Mage.cpp, Warrior.hpp, Mage.hpp dalam format **ZIP**

C++14

soal4.zip

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	100	Accepted	0.00 sec, 3.05 MB

◄ Inheritance, Polymorphism, Virtual

Jump to...

Template & Generic, Exception Handling, STL ►